Boston Bike Crash Explorer — Context Handoff

Prepared 2026-07-22 · for continuing ideation in a fresh Claude conversation

This document captures the project as it stands today: the concept, what's actually been built and verified in the repo, the data sources and their real constraints, and two ideation threads that are still open and unresolved. It's meant as a complete briefing so a new conversation doesn't have to re-derive context — read it as background, not as a fixed scope. The project owner wants this to be a rich, ambitious data visualization piece and is actively exploring how far to take it.

---

1. Concept

Core question: Where and when do reported bike crashes happen in Boston, and does that pattern shift around known policy/infrastructure changes?

Reference / quality bar: jbvordick.github.io/airQualityInteractive (Jillian Vordick, “Pittsburgh Air Quality”). The mechanic being replicated: drag-to-brush a date range on a time-series chart, and a linked map instantly filters to just that range (“Selected dates: X to Y” shown live). A few curated, clickable callout boxes on the map jump to specific interesting date windows — a light narrative layer inside free exploration, not full scrollytelling. Real labeled basemap (street names visible), not an abstract shape.

Portfolio context: built for a Data Visualization Engineer application at Planet (satellite imagery / Earth observation) — but deliberately not framed around Planet explicitly. It's meant to read as a genuine, personally-motivated piece about a real local issue (Boston cycling safety), not a job-application prop.

What it is not, as originally scoped: not a risk/danger ranking (no ridership data to normalize crash counts against), not scrollytelling, not a routing tool. These were the original guardrails — see Section 6 for why they matter even as scope discussions continue.

2. Current Build Status — Verified Today

The following was confirmed directly against the live repo and data file on 2026-07-22, not recalled from memory or an earlier status note.

Repo structure

Svelte + Vite app (`src/App.svelte`, `src/TimeChart.svelte`, `src/CrashMap.svelte`), Python fetch script (`scripts/fetch_crash_data.py`), data file at `public/data/boston_bike_crashes.geojson` (served) and `data/boston_bike_crashes.geojson` (source copy). Four commits on the main branch: initial Svelte/D3/MapLibre scaffold → NaN-to-null GeoJSON fix + real labeled basemap swap → real monthly crash counts and error handling in the chart → repo cleanup.

What's real and working right now

- **Data:** 3,987 real bicycle crash records pulled from Boston's Vision Zero CKAN API, every one with a complete lat/long and timestamp (zero rows dropped in cleaning). Verified date range in the actual file: 2015-01-01 to 2026-06-30.
- **Map:** MapLibre GL JS rendering a real labeled OpenFreeMap Liberty basemap (street names, landmarks visible — not a blank background), with all 3,987 points loaded as a GeoJSON source and rendered as a circle layer.

- **Chart:** D3 area chart of *real* monthly crash counts (grouped from the actual timestamp_ms property on each feature, not placeholder/random data — this was the one open TODO as of the last written status note, and it has since been completed per the commit log). Shows “Count of records: 3,987 reported bike crashes, Jan 2015–present.”
- **Brush-linked filtering (the core mechanic):** confirmed wired end-to-end in code. Dragging a selection on the D3 chart fires a d3-brush 'end' event that sets a shared `selectedRange` variable in the parent component; CrashMap.svelte reacts to that change and applies a MapLibre `setFilter` on the crash-points layer comparing each point's `timestamp_ms` against the selected range. Clearing the brush resets the filter to show all points.
- **Callouts:** two of the four planned Phase 1 callout buttons are implemented and wired to the same `selectedRange` mechanism as a manual brush — “Speed limit dropped 30→25mph (Jan 2017)” and “COVID ridership dip.” The Mass Ave and Comm Ave protected-lane callouts are stubbed in as commented-out placeholders pending confirmed installation dates.
- **Methodology footer:** a visible caveat is already in the UI — explicitly states raw crash counts aren't a danger ranking without ridership data, and that only police/EMS-reported crashes are captured (near-misses and unreported incidents excluded), with a source link to the Vision Zero dataset.

What's not yet built

- Mass Ave / Comm Ave protected-lane callout dates — not yet confirmed from bike-network vintage data (see Section 4.3).
- Street-name search/filter box (Vision Zero data has a `street` field ready to use).
- Deployment — the app has only been run locally via Vite dev server so far.
- Everything discussed in Section 5 (both ideation threads) — neither has been started in code.

---

3. Tech Stack

Layer	Choice	Why
App shell	Svelte + Vite	Lean, fast-loading; Vite for local dev + build
Time chart + brush	D3.js (d3-brush)	Full manual control over the linked-views interaction
Map	MapLibre GL JS	Open-source, no API key or usage billing (vs. Mapbox) — matters for something that mig
Basemap style	OpenFreeMap Liberty	Free, no key required, real street labels
Data prep	Python + pandas + requests	Pull/clean from the Vision Zero CKAN API
Format	Plain GeoJSON, no tiling	~3,500–4,500 points is small enough that PMTiles/tippecanoe would be unneeded comp

---

4. Data Sources

4.1 Primary — Vision Zero Crash Records (Analyze Boston), currently in use

Landing page: data.boston.gov/dataset/vision-zero-crash-records · Resource id e4bfe397-6bfc-49c5-9367-c879fac7401d · CSV dump via CKAN datastore API.

Schema (10 columns, confirmed from the live file): `dispatch_ts, mode_type, location_type, street, xstreet1, xstreet2, x_cord, y_cord, lat, long`. Filtered to `mode_type == 'bike'`. `dispatch_ts` is

when public-safety response was dispatched, not necessarily the crash instant, format YYYY-MM-DD HH:MM:SS+00 (UTC). `x_cord`/`y_cord` are MA State Plane feet and are ignored in favor of lat/long.

Known limitation: no severity/injury field in this file at all — the dataset can show where and when a police/EMS-reported bike crash occurred, never how bad it was. Coverage 2015-01-01 to present, updated monthly. Companion Vision Zero Fatality Records dataset exists separately (BPD-verified fatalities, has crash type) but has not been touched.

4.2 Secondary — MassDOT IMPACT crash data, not yet integrated

Portal: massdot-impact-crashes-vhb.opendata.arcgis.com · ArcGIS REST layer example: gis.crashdata.dot.mass.gov/arcgis/rest/services/MassDOT/MASSDOT_ODP_OPEN_2025/FeatureServer/0.

Adds fields Vision Zero doesn't have: `CRASH_SEVERITY_DESCR, NUMB_FATAL_INJR, NUMB_NONFATAL_INJR`, weather, contributing-factor fields. Data is organized by year (e.g. “2025 Crashes” layer) and offered in CSV/KML/GeoJSON/etc. via the Open Data Hub.

Two facts confirmed by web research this session, not previously verified:

- MassDOT's statewide crash report form was updated on 2024-01-01 (as part of “An Act to Reduce Fatalities”) specifically to add 21 new fields for vulnerable road users (VRU) — pedestrians and cyclists. This means the richest cyclist-specific detail in MassDOT's data exists mainly from 2024 onward, not across the full 2015–2026 window Vision Zero covers. Basic severity/injury-count fields likely have longer history, but the newer VRU-specific fields don't.
- There is no shared crash ID between Vision Zero and MassDOT IMPACT records. Any join between the two datasets has to be inferred — most plausibly a spatial + temporal match (e.g. same lat/long within some radius, same dispatch time within some window) — and that inference will produce matches of varying confidence, plus some Vision Zero bike crashes that don't match anything in MassDOT at all.

4.3 Infrastructure — Boston bike network, not yet integrated

Existing Bike Network dataset: data.boston.gov/dataset/existing-bike-network-2024. Confirmed this session: Boston publishes this as separate yearly snapshot datasets on BostonMaps (2022, 2023, 2024 each have their own distinct dataset page), rather than one dataset with a built-in install-date field per segment. That means an approximate installation year for a given segment (e.g. Mass Ave, Comm Ave) could be inferred by diffing which segments appear in one year's snapshot but not the prior year's — geometry-matching across snapshots, with year-level precision rather than an exact date. This has not been attempted yet; the Mass Ave and Comm Ave callout dates are still open questions (see Section 7).

---

5. Data Pipeline (as built)

- Fetch: pull Vision Zero bike-crash rows via the CKAN CSV dump URL, filter to mode_type == 'bike'.
- Clean: drop rows missing lat/long/timestamp (in practice, zero rows were dropped — every record had complete location and time data, plausibly because this is 911-dispatch-derived data rather than a hand-filled survey).
- Transform: export as GeoJSON with a timestamp_ms numeric property per feature, so the MapLibre filter expression can compare numerically against the brushed range.
- Render: static map first, confirmed points looked right on real streets before wiring any interactivity.
- Chart: D3 area/line chart of monthly crash counts, built from the same GeoJSON file (no separate aggregation pipeline).

- Brush wiring: d3-brush on the chart sets a shared selectedRange prop; MapLibre setFilter on the crash-points layer reacts to it. Confirmed working end-to-end in the current code.
- Callouts: buttons that set selectedRange directly, same mechanism as a manual brush — two of four planned callouts are live.

---

6. Open Ideation — Two Threads in Progress

Both of these are live, unresolved directions the project owner is actively exploring to make this a richer, more differentiated data visualization piece — not settled decisions and not rejected ideas. The facts below were gathered to inform the ideation, not to close it off.

6.1 Idea — pair Vision Zero with MassDOT IMPACT for severity/injury detail

The pitch: Vision Zero gives clean, monthly-updated, geocoded bike-crash points (2015–present) but has no severity field at all. MassDOT IMPACT's crash layers add CRASH_SEVERITY_DESCR, injury counts, contributing factors, weather, and (from 2024 on) detailed vulnerable-road-user fields. Pairing them would let the map/chart show not just where and when crashes happened, but how severe they were — a much richer story than a flat dot map.

What would need to be figured out: the join strategy (no shared ID — spatial/temporal fuzzy match, with explicit confidence tiers), how to visually or textually represent unmatched points and low-confidence matches (the project's existing guardrail is to never fabricate or estimate a data point, which argues for showing match confidence rather than hiding uncertainty), and how to frame the VRU-field time coverage gap (rich detail mainly 2024+, thinner before that) so the severity layer doesn't imply more precision than the underlying data supports across the full time range.

6.2 Idea — corridor before/after explorer with satellite imagery

The pitch: use Boston's dated bike-network layers (2022/2023/2024) as a before/after pivot for specific corridors (Mass Ave, Comm Ave), overlaying crash points on satellite imagery rather than only a street-line basemap. The idea is to go beyond a generic filterable dot-map — which Boston, NYC, and local journalists have already shipped in various forms — by showing a physical, visual before/after at the corridor level: what the street looked like before a protected lane existed, versus after, with crash points layered on each.

What would need to be figured out: exact or year-level installation dates per corridor (the 2022/2023/2024 snapshot datasets exist and could be diffed — see Section 4.3 — but this hasn't been done yet); a satellite/imagery basemap source and its licensing and cost model (Esri World Imagery, Mapbox satellite, or another provider — each has different key/attribution/cost tradeoffs compared to the current no-key OpenFreeMap setup); and the UI shape — whether this lives as an enhancement to the existing map/callout system or as a distinct view/mode within the piece.

---

7. Guardrails That Apply Regardless of Scope

These are framing/accuracy commitments, not scope limits — they hold no matter how large or ambitious the piece grows, because they're about not overclaiming what the underlying data supports.

- Never call a street or intersection “most dangerous” from raw crash counts alone — no ridership/exposure data means no risk ranking, only reported-incident density. Say “reported crashes,” not “danger” or “risk,” unless exposure data is actually joined in.
- Never imply causation between a policy/infrastructure change and a crash-count change shown in the same view — correlation only, with plausible confounders noted (e.g. COVID ridership swings, other concurrent policy changes).

- Never fabricate or estimate a data point. Every number that appears in the UI must be checked against the actual downloaded file, not asserted from memory, training knowledge, or a plausible-sounding guess.
- Only police/EMS-reported crashes are captured in Vision Zero — near-misses and unreported incidents are excluded by definition, and that should stay visible in any methodology note.
- Individual named crashes or fatalities as callouts require careful, deliberate handling (real people, real tragedies) — this was explicitly deferred, not rejected, and should get its own dedicated thought whenever it's picked up.

---

8. Open Questions

- Exact (or year-level, via snapshot diffing) Mass Ave / Comm Ave protected-lane installation dates — not yet confirmed from bike network vintage data.
- Whether and how to pursue the MassDOT severity join (Section 6.1) — join methodology, confidence-tier design, and how to communicate the 2024+ VRU-field coverage gap.
- Whether and how to pursue the corridor/satellite before-after idea (Section 6.2) — imagery source/licensing, and whether it's a standalone mode or an enhancement to existing callouts.
- A short click-through intro (2 screens + breadcrumbs) ahead of the explorer was floated separately and confirmed feasible as a small addition that wraps around the existing explorer without touching the map/chart code — not built, not currently prioritized.
- Street-name search/filter box — planned, not yet built.
- Deployment target and hosting — not yet decided.

---

9. Repo Snapshot

Local path: boston-bike-crashes/ (new repo, clean start — not reused from an earlier Economic-Connectedness repo). Commit history, oldest to newest:

- Initial scaffold: Svelte + D3 brush + MapLibre skeleton
- Fix NaN→null in GeoJSON export, swap to real labeled basemap
- Implement real monthly crash counts and error handling in TimeChart component
- Remove stray Vite temp config files, ignore the pattern going forward

Verified data file stats as of 2026-07-22: 3,987 features in data/boston_bike_crashes.geojson, timestamp range 2015-01-01 to 2026-06-30, sample coordinates fall within expected Boston bounds (e.g. -71.10, 42.33).